



University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik

Software-Architekturen

Klausurzusammenfassung

Sommersemester 2026

Referent: Prof. Dr. Thomas Fuchß

Ersteller: Devin Uyan

Inhaltsverzeichnis

Vorwort	2
1 Software-Engineering	2
1.1 Iterativer Prozess (Larman) und CI/CD	2
1.2 Analyse, Design und das semiotische Dreieck	3
2 Was ist Architektur?	3
2.1 Begriffe nach IEEE 1471	4
2.2 Sichten-Modelle (4+1 und 3+1)	4
3 Wie beschreibt man Architektur?	5
3.1 Struktur, Kommunikation, Deployment	5
3.2 Komponenten (UML)	6
3.3 Konnektoren (UML)	6
4 Funktionale & nicht-funktionale Anforderungen	7
4.1 Typische Quality Attributes & Metriken	7
4.2 Qualität über Szenarien beschreiben (Clements/Bass/Kazman)	7
5 Aufgaben des Architekten: Wählen, Bewerten, Testen	8
5.1 Der Architektur-Zyklus	8
5.2 Die vier Schritte im Detail	9
5.3 SAAM – Software Architecture Analysis Method	9
6 Referenzarchitekturen & Designprinzipien	10
6.1 Patterns	10
6.2 SOLID (Robert C. Martin)	10
6.3 Domain-Driven Design (Eric Evans)	10
6.4 Vom Chaos zur Struktur – die drei Fundamente	11
7 Layers – Adapters – Onions	11
7.1 Layers (Schichten)	11
7.2 Layers vs. Tiers	12
7.3 Application Server vs. Microservices	13
7.4 Onion / Hexagonal – Wer nutzt wen?	13
7.5 Ports und Adapter (Hexagonal, Cockburn)	14

8	Blackboard und KI	14
8.1	Struktur	15
8.2	Realisierung & Bewertung	15
9	Pipes and Filters	15
9.1	Aktive vs. passive Filter, Pull/Push	16
9.2	Varianten & Bewertung	16
9.3	Mediator-Pattern (Event-Driven Architecture)	16
10	Middleware – gestern, heute, morgen	17
10.1	Abstraktionsebenen	17
10.2	Kommunikation: OOM vs. REST/gRPC	17
10.3	MOM – Entkopplung durch Nachrichten	18
10.4	Gedanken zur Verteilung	18
11	Patterns für den Business Layer	19
11.1	GRASP – das Herzstück des Designs (Larman)	19
11.2	Die drei Business-Layer-Patterns	19
11.3	Aufwand im Verhältnis zur Komplexität (Fowler)	20
12	Zwischen Architektur und Design: Interfaces	20
12.1	Proxy	20
12.2	Bridge (Brücke)	21
12.3	Adapter	21

Vorwort

Diese Zusammenfassung deckt die Vorlesung *Software-Architekturen* ab. Sie verdichtet die zentralen Begriffe, Sichten, Architektur- und Entwurfsmuster sowie die Middleware-Landschaft auf das für die Klausur Wesentliche. Alle Abbildungen wurden als eigene TikZ-Grafiken bzw. Tabellen neu erstellt.

Merke

Roter Faden der Vorlesung: **Nicht-funktionale Anforderungen** (Quality Attributes) treiben die Architektur. Architektur ordnet ein System in **Komponenten** (*was* tut etwas) und **Konnektoren** (*wie* kommunizieren sie). Muster sind bewährte Lösungen für wiederkehrende Konflikte zwischen widerstreitenden Anforderungen.

1 Software-Engineering

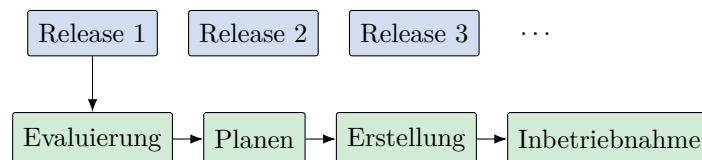
Software-Engineering ist die *Technologie- und Management-Disziplin* für die *systematische Erstellung und Wartung* großer Software-Systeme unter Verwendung bewährter Vorgehensweisen, Prinzipien, Methoden und Werkzeuge (Broy/Rombach). Ingenieure bauen Artefakte auf Basis von Theorien und Methoden – unter finanziellen und organisatorischen Constraints (Sommerville).

Merke

Systematik ist unabdingbar in: **Prozess, Analyse, Design** und **Test**. Allen modernen Prozessmodellen gemeinsam: **agiles, iteratives und risikoorientiertes Vorgehen**.

1.1 Iterativer Prozess (Larman) und CI/CD

Larman: in vier Schritten zum Release – *Evaluierung (Phase I) → Planen → Erstellung (Phase II) → Inbetriebnahme*; mehrere Releases iterativ.



Modern ersetzt **CI/CD** die Erstellungs- und Inbetriebnahme-Phase. Die zentralen Herausforderungen (Analyse, Design, Implementierung) bleiben jedoch gleich.

Konzept	Bedeutung
CI (Continuous Integration)	Arbeit an gemeinsamer Code-Basis mit eigenem Feature-Branch; Integration per <i>Pull-Request</i> : automat. Bauen + Testen, abgesichert durch Quality Gate . Pull-Request $\hat{=}$ <i>Inversion of Control</i> : man kann nicht selbst in den Main-Branch pushen, der Code wird gezogen $\Rightarrow$ nur geprüfter Code wird integriert.
CD (Cont. Delivery)	Software ist <i>jederzeit auslieferbar</i> ; Deployment erfolgt aber weiterhin nach manueller Freigabe .
CD (Cont. Deployment)	Nach erfolgreichem Build folgt <i>automatisiertes</i> Deployment – manuelle Freigabe entfällt.
CI-Prozess sichert die Code-Qualität, CD-Prozess garantiert Auslieferung/Bereitstellung des korrekten Codes. Typischer Zyklus: Pull-Request $\rightarrow$ CI-Pipeline (Checkout, Build,	

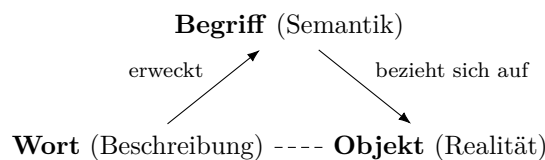
Unit-/Integrationstest, Analyse) → Quality Gate → Merge + Container-Build (Image versioniert) → Deploy (Manifest aktualisiert, Image in Registry, Zielplattform z. B. Kubernetes *zieht es*).

Zielformen: CaaS (Container/Orchestrierung: Kubernetes, ECS), PaaS (Azure, GCP), SaaS-Hosting (Netlify, Vercel), IaaS (VMs: VMware, EC2), App Stores.

1.2 Analyse, Design und das semiotische Dreieck

Phase	Inhalt
Analyse	Das Problem verstehen & beschreiben, sodass ein korrekter Entwurf möglich wird (Use-Case-, Aktivitäts-, Klassen-, Sequenzdiagramme). Kurz: <i>Welche Schnittstellen hat das System?</i>
Design	Die Lösung beschreiben, sodass die Anforderungen im Sinne der Stakeholder umgesetzt werden. Systementwurf (Bindeglied: Aufbrechen in Teilsysteme, Strategien für Steuerung/Datenhaltung) und Objektentwurf (Verantwortlichkeiten zuweisen). Kurz: <i>Wie wird die Aufgabe realisiert?</i>

Beide Phasen treffen **primäre Architekturentscheidungen**. Das **semiotische Dreieck** (Ogden & Richards) erklärt das Grundproblem von Analyse und Design:



Merke

Frage von Analyse & Design: Stimmt „*Das, was der Kunde will*“ mit „*Das, was der Entwickler glaubt bauen zu müssen*“ bzw. „*Das, was gebaut wird*“ überein? ⇒ **Konvergenz durch Iteration!**

2 Was ist Architektur?

Architektur ist das **zentrale Glied** in der Systematik des Vorgehens.

Merke

Architektur (IEEE 1471-2000): *the fundamental organization of a system embodied in its components, their relationships to each other, and to the environment, and the principles guiding its design and evolution.*

Essentials: Die Architektur ist ein *Bauplan* für die Organisation des Gesamtsystems vom Entwurf über die Entwicklung bis zum Betrieb. Sie erfasst die *signifikanten* statischen und dynamischen Aspekte: Struktur der Subsysteme/Komponenten, deren Interfaces, Verbindungen, Interaktion, Verantwortlichkeiten und Zusammenarbeit (Aspekte: Verwendung, Leistung, Technologie, Realisierung, Konfiguration).

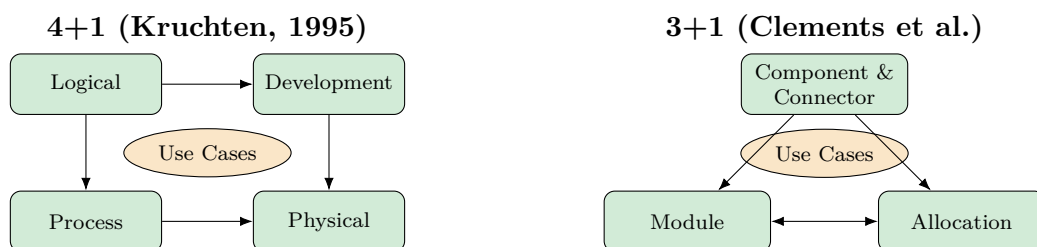
2.1 Begriffe nach IEEE 1471

Begriff	Bedeutung
Architecting	Tätigkeiten: Definieren, Dokumentieren, Pflegen, Verbessern, Zertifizieren der korrekten Umsetzung.
Architect	Person/Team/Organisation, verantwortlich für die Systemarchitektur.
Architectural Description (AD)	Sammlung von Produkten zur Dokumentation einer Architektur.
System	Sammlung von Komponenten zur Erfüllung bestimmter Funktion(en).
Stakeholder	Individuum/Team/Organisation mit Interessen/Concerns am System (Designer, Entwickler, PM, Kunde, ...).
Concern	Ziel/Wunsch: zentrale Anforderungen, unabdingbare Constraints, Absichten, vage Ideen – schwer zu entdecken, aber erfolgsentscheidend.
View	Darstellung des Gesamtsystems aus Perspektive zusammenhängender Concerns.
Viewpoint	Spezifikation der Konventionen zur Konstruktion/Nutzung einer View (Schablone).

Zusammenhang: *System* hat *Architektur*, beschrieben durch *AD*; Stakeholder verfolgen Concerns, die von der AD identifiziert werden; ein Viewpoint definiert Views, die aus *Models* bestehen und die AD strukturieren.

2.2 Sichten-Modelle (4+1 und 3+1)

Jede(r) View(point) beschreibt die Architektur aus einer anderen Sicht.



4+1 View	Fokus	Inhalt / Diagramme
Logical	Anwendersicht	Struktur & Zusammenarbeit der Bausteine (funktionale Anf.); Class-, Sequence-, Component-Diagramme.
Process	Nebenläufigkeit	Verteilung auf selbstständige Entitäten, Kommunikation/Synchronisation; NFR: Performance, Verfügbarkeit, Fehlertoleranz. Activity-, Sequence-, Composite-Structure-Diagr.
Physical	reale Struktur	Verteilung der Baugruppen auf HW-Knoten; NFR: Skalierbarkeit, Ausfallsicherheit, Performance. Deployment-Diagramme.
Development	Entwicklersicht	Pakete/Module/Bibliotheken, Konfigurationsmgmt. Package-, Class-, Deployment-Diagr.
Use Case (das +1)	Anforderungen	funktionale Anf. als Use Cases/Szenarien – <i>Maß, an dem sich die Architektur orientiert.</i>

3+1: Da sich Logical/Development/Process nur schwer trennen lassen, fasst Clements zusammen:

3+1 View	Beschreibt	Stakeholder
Component & Connector	was das System „macht“ (dynamisch, Daten-/Kontrollfluss); Komponenten = Objekte/Prozesse/Threads, Konnektoren = Shared Memory/Sockets/JDBC/ESB	Architekten, Entwickler, Kunden, Anwender
Module	Modularisierung: Strukturen, Klassen, Interfaces, Pakete & Beziehungen	Architekten, Entwickler
Allocation	Bibliotheken, Konfig., Deployment (SW→HW), Implementierung (SW→Dateien), Arbeitspakete (SW→Teams)	alle

Klausur-Fokus

Zentrale Gesichtspunkte der Architekturbeschreibung: Modularisierung und *Struktur*, Modularisierung und *Kommunikation*, Modularisierung und *Deployment* (Konfiguration, Integration, Arbeitsteilung).

3 Wie beschreibt man Architektur?

3.1 Struktur, Kommunikation, Deployment

Divide et impera: Jede komplexe Aufgabe wird in kleinere, beherrschbare *Komponenten* zerlegt; jede übernimmt eine dedizierte Teilaufgabe, ihre strukturierte Zusammenarbeit löst das Problem. Die Architektur definiert diese Komponenten, strukturiert sie und beschreibt ihre Interaktion.

Merke

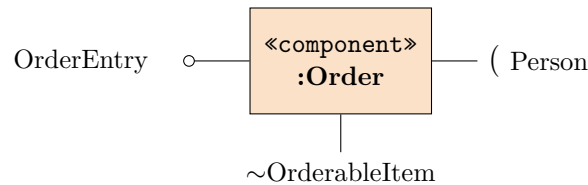
Abhängigkeit: Zwei Komponenten *A, B* sind abhängig, gdw. eine Änderung in einer eine Änderung in der anderen nach sich zieht. **Ziel: Minimierung der Abhängigkeiten** – z. B. durch eine *Zwischenschicht*.



Je mehr Abhängigkeiten, desto aufwendiger werden: Einbau/Änderung von Funktionalität, Testen, Bauen – und v.a. die **Abstimmung zwischen den Teams**.

3.2 Komponenten (UML)

Komponente (Szyperski): ein *kontextabhängiger Softwarebaustein mit definierten Schnittstellen*. (Meyer: modulare Einheit, die von Clients verwendbar ist, eine zur Nutzung ausreichende Schnittstelle besitzt und nicht an einen vorgegebenen Client gebunden ist.)



Element	Bedeutung
Provided Interface (Lollipop $\ominus$)	Schnittstelle, die die Komponente <i>anbietet</i> .
Required Interface (Socket $\ominus$)	Schnittstelle, die die Komponente <i>benötigt</i> (erwartet, dass eine andere sie bereitstellt; $\sim$ = konjugiert).
Port	dedizierter Interaktionspunkt mit der Umgebung, definiert durch die Menge der bereitgestellten und benötigten Interfaces.

3.3 Konnektoren (UML)

Konnektor-Art	Funktion
Delegate-Connector	verbindet einen äußeren Port mit einem inneren (Weiterleitung in zusammengesetzten Komponenten).
Assembly-Connector	verbindet <i>required</i> mit <i>provided</i> Interface zweier Komponenten ($\ominus \subset -$).
Kompositionsstruktur	dedizierter Konnektor zwischen zwei Ports.
Assoziation(sklasse)	Konnektor im Klassendiagramm (z. B. Buffer zwischen Scanner und File).

Merke

Konnektoren sind explizit zu modellieren (implizite vermeiden!). Sie bekommen eindeutige Bezeichnung + eigene Spezifikation: *welche* Schnittstellen, *wie* (synchron/asynchron), *welches Protokoll* (FIFO, LIFO, Producer/Consumer), *wie umgesetzt* (JSON, RPC, Java-RX, Shared Memory) und *welche NFR*.

Goldene Regel: Ein Konnektor *verändert die Daten nicht* – es wird keine Information hinzugefügt oder entfernt (Semantik(A) $\Leftrightarrow$ Semantik(A')). Die Semantik ist aber kontextabhängig (mobile Übertragung erfordert oft Adaption).

ADLs (Architecture Description Languages, z. B. ACME) sind spezifische Modellierungssprachen: Austauschformat (interoperable Werkzeuge), Framework und einfache Sprachkonstrukte für Komponenten + Konnektoren (mit Rollen, Ports, **Attachments**).

4 Funktionale & nicht-funktionale Anforderungen

Anforderungsart	Bedeutung
Funktionale Anf.	legen den <i>Umfang</i> der Lösung fest – <i>was</i> man bekommt. Objektiv: Funktionalität ist vorhanden oder fehlt.
Nicht-funktionale Anf. (Constraints)	definieren die <i>Qualität</i> der Lösung – <i>wie gut</i> das ist, was man bekommt. Subjektiv: Qualität ist für jeden etwas anderes.

Merke

Nicht-funktionale Anforderungen **sind Software Quality Attributes**. Um sie messbar/bewertbar zu machen, braucht man **Metriken**.

Verallgemeinert: Alles, dessen Erfüllungsgrad man mit *Ja/Nein* beantworten kann, ist „funktional“; alles, worüber man streiten kann, ist „nicht-funktional“. Über Metriken werden NFR in objektive (pseudo-)funktionale Anforderungen verwandelt.

Beispiel Security: „sensible Daten vertraulich behandeln“ ist nicht-funktional; „nach 3 Fehlversuchen Account 5 min sperren“ bzw. „personenbez. Daten mit AES-256 verschlüsseln“ ist funktional & überprüfbar. Auch *technische* Anforderungen (Sprache, Plattform, HW, Framework) können je nach Formulierung funktional oder nicht-funktional sein.

4.1 Typische Quality Attributes & Metriken

Quality Attribute	Metrik(en)
Performance (Leistung)	Antwortzeit (Latenz); Durchsatz (Transaktionen/s)
Reliability (Zuverlässigkeit)	Fehlerrate; mittlere Betriebsdauer (MTBF)
Scalability (Skalierbarkeit)	Skalierungszeit (Anpassung der Ressourcen)
Usability	–
Maintainability (Wartbarkeit)	Ø Zeit zur Fehlerbehebung

$$P = \frac{\Delta W}{\Delta t} \text{ (Leistung = erbrachte Arbeit pro Zeit).} \quad \text{Verfügbarkeit} = \frac{\text{Gesamtzeit} - \text{Downtime}}{\text{Gesamtzeit}},$$

$$\text{MTBF} = \frac{\text{Gesamtzeit}}{\text{Anzahl Fehler}}.$$

4.2 Qualität über Szenarien beschreiben (Clements/Bass/Kazman)

Merke

Qualität muss man wie Funktionalität *definieren, bewerten, messen, testen* – sie ist nicht NULL oder EINS! **Wie? Mit Hilfe von Szenarien.** Ein **Szenario** ist die konkrete Ausprägung eines Use Cases (Interaktion Nutzer ↔ System in einer Situation); ein **Anwendungsfall** ist eine Menge von Szenarien mit gemeinsamem Ziel.

Sechs Elemente eines Quality-Attribute-Szenarios:

Element	Bedeutung
Quelle (Source of Stimulus)	Akteur, der ein Ereignis auslöst.
Stimulus	definiertes Ereignis, auf das das System reagieren muss.
Artefakt	betroffener Teil des Systems.
Environment	Ausprägung/Lastzustand, unter dem betrachtet wird.
Antwort (Response)	Systemverhalten unter dem Stimulus.
Maße/Metriken (Response Measure)	Angaben zur späteren Bewertung.

Beispiel Verfügbarkeit & Modifizierbarkeit (Szenario-Ausprägungen)

Verfügbarkeit (Ausprägung der Zuverlässigkeit): Concerns sind Downtime (geplant: Wartung/Upgrade; ungeplant: nicht maskierbare HW-/SW-Ausfälle), *Time to Repair*, *Time to Recover*. Quelle: fehlerhafte Komponente/Konnektor, externer Störer; Stimulus: Ausfall, falsch formatierte Nachricht; Response: Fehler erkannt/weitergeleitet/bereinigt; Maß: Wahrscheinlichkeit/Häufigkeit/Dauer.

Modifizierbarkeit: Aufwand, eine Anwendung an neue Anforderungen anzupassen (*Was/Wer/Wann/Wo* verursacht Änderung?). Maße: Kosten (€), Zeit (Manntage), Umfang (LOC).

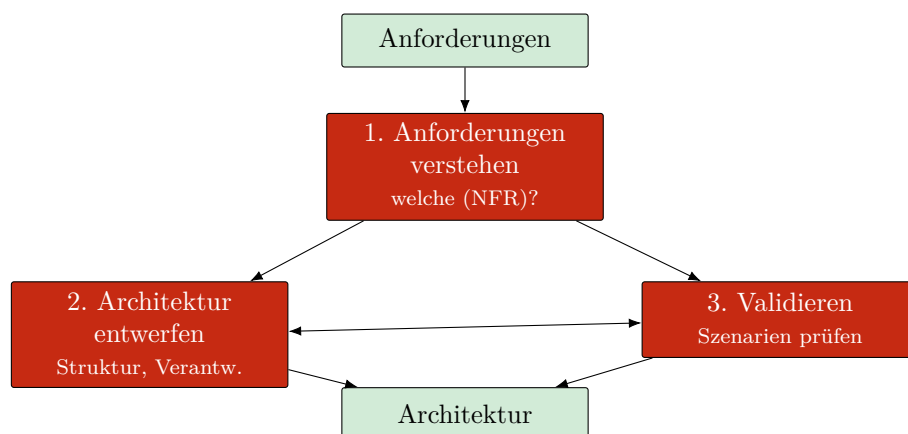
Scalability: Verhalten, wenn sich eine Dimension des Problemraums ausdehnt – Lastverhalten (Antwortzeit, Durchsatz), simultane Verbindungen, Datengröße, Anzahl Installationen. Problem: Skalierbarkeit ist anfangs oft keine „offizielle Anforderung und Tests sind teuer $\Rightarrow$ *nicht skalierbare Architekturen vermeiden, auf Bewährtes zurückgreifen*.

Klausur-Fokus

Eine gute Architektur muss **allen – oft widersprüchlichen – Anforderungen** gerecht werden. Eine echte Herausforderung: Performance $\leftrightarrow$ Reliability, Modifizierbarkeit $\leftrightarrow$ Zeitplan, usw.

5 Aufgaben des Architekten: Wählen, Bewerten, Testen

5.1 Der Architektur-Zyklus



Merke

Die Architekturentwicklung ist **kein monolithischer Schritt**: agil, inkrementell, iterativ, risikoorientiert. Sie ist *nicht losgelöst* von der Umsetzung der funktionalen Anforderungen – Architektur und Use Cases beeinflussen sich gegenseitig und reifen gemeinsam [RUP]. In den *ersten Sprints* entsteht der Kern (prototypisch), nachfolgende Sprints stabilisieren ihn; Anpassung durch permanentes **Refactoring**.

5.2 Die vier Schritte im Detail

1. Requirements erfassen. Aus funktionalen + sonstigen Anforderungen die *NFR extrahieren* (Quality Attributes & System Constraints). Tabellarisch festhalten:

Kategorie	Beispiel-Architekturanforderung
Performance	Antwortzeit < 1 s (flüssiges Arbeiten).
Security	alle Benutzer eindeutig identifiziert; jeder Kanal außerhalb des Back-Ends verschlüsselt.
Verfügbarkeit	24×7×365 bei 95 %; MTBF ≥ 2 Wochen.
Modifizierbarkeit	Anbindung mobiler Endgeräte leicht möglich.
Techn. Constraint	App-Server Tomcat X.Y; Sprache echtzeitfähig.
Business Constraint	Time-to-Market (Ende des Jahres); Budget ≤ 100.000 €.

2. Priorisieren. Wenige, klare Stufen: **Hoch** (muss – primäre Aufgabe der Architektur), **Mittel** (sollte – in späterem Release erfüllbar anlegen), **Niedrig** (nice-to-have – nicht mutwillig verletzen). Probleme entstehen, sobald sich Anforderungen *widersprechen* (z. B. mobile Anbindung vs. Termin).

3. Entwurf. Wahl der primären Architektur (Client-Server, N-Tier, ...), Zerlegen in Komponenten, Definieren der Konnektoren. *Erfordert Wissen und Erfahrung.*

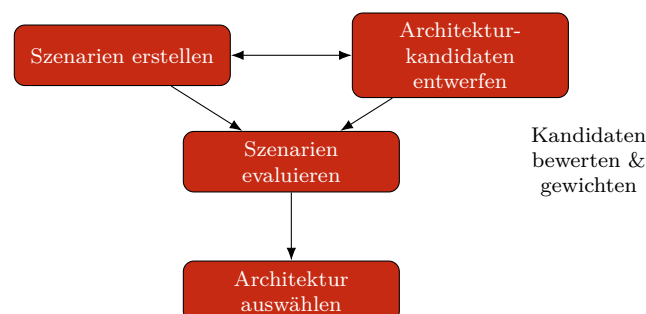
4. Validieren. Szenarien durchspielen, Prototypen bauen.

Klausur-Fokus

Achtung beim Validieren: (1) Die Architektur beruht auf *zentralen* Szenarien – zum Testen müssen sie *variiert* werden, sonst findet man kaum Fehler. (2) Prototypen sind für *Skalierungs- und Performance-Fragen* nur bedingt geeignet. Im agilen Vorgehen kann am „echten Produkt“ der ersten Sprints validiert werden.

5.3 SAAM – Software Architecture Analysis Method

Szenariobasierte Bewertung mehrerer Architekturkandidaten:



6 Referenzarchitekturen & Designprinzipien

6.1 Patterns

Merke

Ein **Pattern** ist eine *Relation zwischen dem Kontext (Situation), dem System widerstreitender Kräfte (Anforderungen) und der Beschreibung einer bewährten Lösung* für diesen Konflikt. Das Problem: naheliegende Realisierungen unterschiedlicher Anforderungen sind nicht vereinbar – **Patterns lösen diese Konflikte.**

Nutzen: Wiederverwendung bewährter Architekturen (→ bessere Entwürfe), Dokumentation von Entscheidungen (→ besseres Verstehen), Standardvokabular (→ Wissensaufbau/-erhaltung). Qualität liegt in der Lösungsbeschreibung. Patterns sind **konstruktiv** (liefern Instruktionen), **abstrakt** (breites Spektrum) und **stabil** (erfassen den stabilen Kern) – langlebig, keine Mode.

Aufgaben von Architekturmustern: Vorlagen zur *Strukturierung*

(Component & Connector View: sinnvolle Zerlegung, Aufgabenverteilung, Konnektorwahl) und zur *Implementierung* (Module View: innere Struktur, zentrale Klassen/Interfaces, Kommunikation). Sie liefern Lösungen für ganze Systemklassen (verteilte, interaktive, adaptierbare Systeme).

Merke

Die Auswahl der richtigen Architekturmuster ist durch die nicht-funktionalen Anforderungen bestimmt!

6.2 SOLID (Robert C. Martin)

Ziel: Wartbarkeit, Erweiterbarkeit, Wiederverwendbarkeit, geringe Kopplung.

Prinzip	Aussage
S Single Responsibility	Jede Klasse/Komponente hat nur <i>eine</i> Verantwortlichkeit.
O Open/Closed	Offen für Erweiterung, geschlossen für Modifikation.
L Liskov Substitution	Subklassen ersetzen Basisklassen problemlos.
I Interface Segregation	Viele spezifische Interfaces statt weniger „God-Interfaces“.
D Dependency Inversion	Abhängigkeit von <i>Abstraktionen</i> , nicht von Implementierungen.

6.3 Domain-Driven Design (Eric Evans)

Die **Fachdomäne** ist der Ausgangspunkt der Zerlegung – nicht die technische Realisierung. Erfordert: enge Zusammenarbeit fachlicher & technischer Experten, eine *gemeinsame Sprache*, präzise Modellierung der Fachkonzepte, Isolation/Abgrenzung der Aufgaben. Die Architektur spiegelt die *fachliche* Struktur wider.

Konzept	Bedeutung
Isolating the Domain	klar abgegrenzte fachliche Teilbereiche (spez. Datenmodelle, Logik, Schnittstellen) ⇒ hohe Kohäsion, lose Kopplung .
Model expressed in Software	Domänenbeziehungen spiegeln sich in Packages/Klassen/Objekten wider (Wer kennt wen? Wer macht was?).
Lifecycle of a Domain Object	<i>Application Scoped</i> (unabh. von Anwender/Aufgabe), <i>Session Scoped</i> (anwender-/aufgabenabh.), <i>Request Scoped</i> (nur für eine Funktion).

Intuition – einfach erklärt

SOLID ist *technisch* (wie gute Klassen/Module aussehen), DDD ist *fachlich* (wie man am Problem schneidet). Beide ergänzen sich (hohe Kohäsion, lose Kopplung). Man sollte sie aber nicht dogmatisch *immer* anwenden – der Aufwand muss zur Komplexität passen.

6.4 Vom Chaos zur Struktur – die drei Fundamente

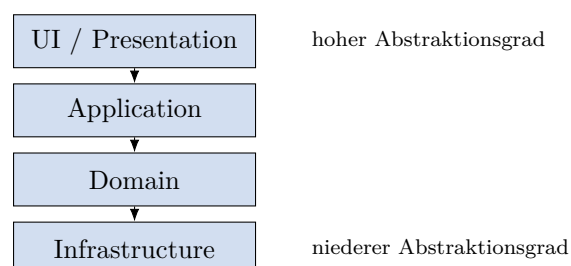
Fundament	Idee
Layers	Abstraktionsebenen schaffen Verantwortlichkeiten; Hierarchien organisieren Aufgaben.
Blackboard	Basis kooperativer Problemlösung: hochspezialisierte Komponenten erarbeiten koordiniert eine Lösung auf geteilter Datenbasis.
Pipes & Filters	Trennen Konnektoren (Pipes) von Komponenten (Filter), ermöglichen kontinuierliche Datenverarbeitung.

7 Layers – Adapters – Onions

7.1 Layers (Schichten)

Merke

Ziel der **Schichtung**: eine Anwendung so strukturieren, dass Teilaufgaben *unterschiedlicher Abstraktionsebenen* getrennt werden. Idee: modular statt monolithisch – konzeptionell Verschiedenes auch getrennt realisieren. Eine klare **Hierarchie** muss erkennbar sein.



Regeln: höhere Schichten verwenden tiefere – **niemals umgekehrt!** Jede Schicht ist *Client* (für tiefere) und *Server* (für höhere). **Partition** = zusätzliche *horizontale* Zerlegung auf gleicher Ebene; Partitionen sollten sich nicht gegenseitig nutzen. Schnittstellen müssen *stabil* (standardisiert) sein, damit spätere Änderungen sich nicht fortpflanzen und Schichten austauschbar bleiben. **Blackbox-Zugriff** (über Interface) ist dem **Whitebox-Zugriff** stets

vorzuziehen; das Durchbrechen der Hierarchie sollte die Ausnahme sein.

Dynamik: *Top-Down delegieren* (N+1 nutzt N per Methodenaufruf) und *Bottom-Up informieren* (Ereignisse aus N werden N+1 gemeldet). Herausforderung: **N darf N+1 nicht kennen** $\Rightarrow$ Callbacks/Observer; die Schnittstellen zur Informationsweitergabe werden in der tieferen Schicht definiert.

Bewertung	Details
Vorteile	Wiederverwendung; Änderungen betreffen idealerweise nur Nachbarschichten; austauschbar.
Nachteile	richtige Granularität schwer; Änderungen eskalieren oft nach oben; Effizienz (unnötiges Kopieren, niedrigere Schicht arbeitet „für niemanden“). Im selben Prozessraum ggf. auf gleichen Daten arbeiten (<code>typedef TypeL1 TypeL2</code>).

7.1.1 Die drei klassischen Schichten

Schicht	Aufgabe
1. Präsentationsschicht	Zugriff von außen (Fenster, Dialoge, Web-Seiten).
2. Logikschicht	eigentliche Lösung (Business-/Geschäftslogik), weiter aufgeteilt.
3. Persistenzschicht	Anbindung an Drittsysteme (DB, Dateisystem, Netzdienste, Mail).

7.2 Layers vs. Tiers

Merke

Layers = *logische* Organisation (wo/wie sie physisch laufen ist offen). **Tiers** = *physische* Schichten – wo der Code zur Ausführung kommt (Deployment). Tiers sind Teil der *Physical/Allocation View*.

Tier (physisch)	enthält Layer (logisch)
Client Tier	Presentation (Web Browser, Mobile App)
Business Tier (App-Server)	Presentation (SSR/API-Controller), Logic (Business/Process), Persistence (OR-Mapping, DB-Driver)
EIS Tier (DB-Server)	Datenbanksystem

Bei Microservices: *Client Tier* – *Gateway Tier* (API-Gateway) – *Service Tier* (je Service: Service-API, Business/Process Logic, Database per Service).

7.3 Application Server vs. Microservices

	Application Server	Microservices
Grundidee	klassische Schichten	fachliche Trennung (DDD)
Geschäftslogik	zentraler Business Tier	dezentrale Services
Datenhaltung	zentraler EIS Tier	Database per Service
Kopplung	eher stark (zentraler Tier)	gering (dezentral)
Skalierung	Tier-weise (EIS meist nicht repliziert)	Service-weise (Konsistenz wird komplex)
Deployment	Tier-weise	Service-spezifisch (CD)
Technologie	einheitlich	auch heterogen
Kommunikation	Netzwerk + lokal in der Logik	nur Netzwerk

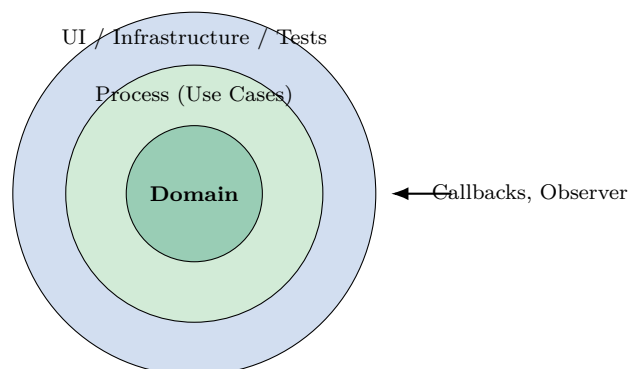
App-Server: physischer Kern (Business Tier) mit Komponenten- und *Container*-Modell; die Laufzeitumgebung stellt Dienste bereit (Multi-Threading, Security, Transaktionsmanagement) ⇒ Entwicklung fokussiert auf die Anwendung. Kommunikation zwischen Tiers ist synchron und gerichtet.

Microservices: API-Gateway (Single Point of Entry: Security, Routing) + Microservices (eigene Logik & eigene Daten). Herausforderung: komplexe, service-übergreifende Anfragen → lokale Transaktionen (*Saga*, *CQRS*).

7.4 Onion / Hexagonal – Wer nutzt wen?

Merke

Stabilität entsteht, wenn man sich *nicht* ständig anpassen muss. **Der Kern (Domäne) ist stabil, der Rest wird darum herum gebaut.** Damit kehrt sich die Abhängigkeit um: nicht die Logik nutzt die Infrastruktur, sondern die *Peripherie hängt vom Kern ab*.



Onion (Palermo): von innen nach außen *Domain Model* → *Process Layer (Use Cases)* → *Application Services (Transport)* → *Peripherie/DB/Frameworks*. Äußere Schichten kennen innere, nicht umgekehrt; Kommunikation nach außen über Callbacks/Observer.

7.5 Ports und Adapter (Hexagonal, Cockburn)

Merke

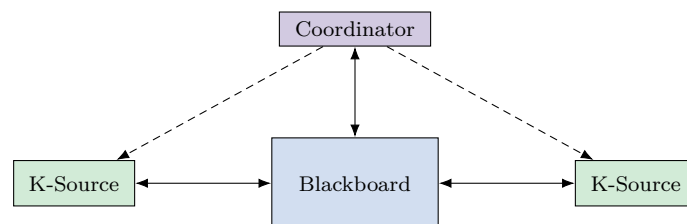
Ziel: Aufgaben bleiben in der zuständigen Schicht und werden nicht „verschmiert- *keine Business-Logik in der Präsentationsschicht*. Komponenten stellen Funktionalität über **Ports** bereit und werden über **Adapter** verbunden.

Variante	Bedeutung
Port als Vererbung	<code>interface PortA extends IAda, IBob</code> – man legt sich auf eine Implementierung fest, <i>der Port ist das Interface</i> .
Port als Aggregation	<code>interface PortA { IAda iAda(); IBob iBob(); }</code> – man <i>bezieht</i> das Interface über den Port (flexibler). Realisierung als <i>Fassade</i> .
Adapter	konkrete UI/Technologie nutzt den Port (Adapter-Pattern: <code>Client → Interface ← A ◊ B(PortA)</code>).

8 Blackboard und KI

Merke

Blackboard: Eigenständige Komponenten (*Knowledge Sources*) erarbeiten gemeinsam und *koordiniert* eine Lösung auf einer **geteilten Datenbasis**. Eine *Kontroll-/Moderator*-Komponente bestimmt die Zusammenarbeit; eine direkte Zusammenarbeit der Wissensquellen findet nicht statt.



Einsatz: Probleme *ohne* deterministische Lösung (historisch: Expertensysteme/KI). Es gibt viele Strategien für Teilprobleme; suboptimale Lösungen reichen oft; man arbeitet mit unscharfem Wissen. Randbedingungen: vollständiges Durchsuchen inakzeptabel (exponentiell), Gebiet noch nicht ganz verstanden (experimentell), Teilprobleme verschieden gelöst (explorativ). Zwischenergebnisse & Hypothesen werden aufgestellt, validiert, verworfen, weiterverfolgt – in einheitlichem Format.

8.1 Struktur

Komponente	Aufgabe
Blackboard	verwaltet Daten (Hypothesen unterschiedlicher Qualität + Kontrolldaten); Repository, auf das alle Wissensquellen zugreifen. <code>inspect()</code> , <code>update()</code> .
Knowledge Source	unabhängige Komponente; evaluiert eigene Anwendbarkeit (<code>execCondition</code>), berechnet & aktualisiert (<code>execAction/updateBlackboard</code>). Vorwärts- (spezifischere Hypothesen ableiten) / Rückwärtsverkettung (validieren/verwerfen).
Control	überwacht das Blackboard, wählt & aktiviert Wissensquellen (<code>loop</code> , <code>nextSource</code>).

Ablauf (Control-Loop): `nextSource` → `execCondition` (mehrere Quellen prüfen Anwendbarkeit gegen das Blackboard) → Auswahl → `execAction/updateBlackboard` → wiederholen bis Problem gelöst. **Repository-Variante:** Blackboard *ohne* interne Kontrolle – die Kontrolle liegt bei externen Anwendern/Anwendungen (z. B. IDE mit Plugins).

Modern (KI): Wissensquellen als *Adapter für LLMs*; Orchestrierung über Frameworks wie LangChain/LangGraph.

8.2 Realisierung & Bewertung

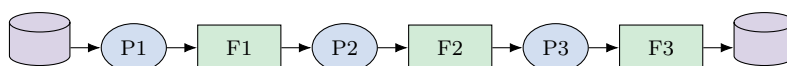
Schritte: (1) Problem definieren, (2) Lösungsraum strukturieren, (3) Lösungsprozess in Schritte aufspalten, (4) spezialisierte Wissensquellen definieren (auch menschliche Experten), (5) Blackboard- Vokabular + Konverter, (6) Kontrollkomponente (Bewertungs-/Auswahlstrategie), (7) Wissensquellen implementieren (Bedingungs-/Aktionsteil, keine Annahmen über andere; ggf. Regelsysteme, LLMs).

Bewertung	Details
Vorteile	experimentierfreundlich; Änder-/Wartbarkeit (strikte Trennung Quelle/Kontrolle/Daten); wiederverwendbare Wissensquellen; fehlertolerant & stabil.
Nachteile	schwer zu testen (nicht nachvollziehbar, unscharfe Daten); <i>keine Lösungsgarantie</i> (oft nur ein Prozentsatz gelöst).

9 Pipes and Filters

Merke

Pipes & Filters strukturiert Systeme, die Daten *Schritt für Schritt* verarbeiten. Jeder Bearbeitungsschritt ist in einem **Filter** gekapselt; die Konnektoren heißen **Pipes**. Durch Standardisierung von Filtern & Konnektoren entstehen leicht adaptierbare, konfigurierbare Systeme.



Übergeordnete Ziele: Workflow-Verarbeitung; Erweiterung durch Austausch/Einfügen von Komponenten. Daraus folgt: Filter bleiben *eigenständig*; der Transport ist *standardisiert* (festes Protokoll); **Datenfluss = Kontrollfluss**; Filter arbeiten **seiteneffektfrei**. Jeder Filter hängt nur von den Ausgaben des vorherigen ab. Filter sollten *parallel* arbeiten.

9.1 Aktive vs. passive Filter, Pull/Push

Filtertyp	Verhalten
Passiver Filter	wird von außen ausgelöst – <i>Pull</i> (Nachfolger entnimmt Daten, <code>read()</code>) oder <i>Push</i> (Vorgänger übergibt, <code>send()</code>).
Aktiver Filter	holt sich selbst Daten aus der Pipe / schreibt sie (Push-Pull-Kombination möglich, läuft als eigener Thread).

Implizite Pipes (direkter Methodenaufruf/RPC): kaum Parallelität, Flexibilität geht verloren. **Explizite Pipes** (aktive Komponenten: Message Queues, Shared Memory, rabbitMQ): echte Parallelität, definierte Austauschformate (JSON, ASN.1, XML), aber hoher Kommunikationsaufwand.

9.2 Varianten & Bewertung

Tee-and-Join: Filter mit mehreren Ein- (Join) und Ausgängen (Tee) $\Rightarrow$ gerichteter Graph mit Rückkopplung/Aufspaltung; Komplexität steigt, Synchronisierung nötig. **Globale Daten:** statt Daten durchzureichen, gemeinsame Container nutzen und nur Referenzen weitergeben.

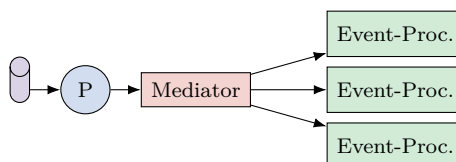
Bewertung	Details
Vorteile	keine Zwischendateien nötig; Tees ermöglichen Kontrolle; Flexibilität (Filter austauschen/umordnen/wiederverwenden); Effizienz durch Parallelität aktiver Filter.
Nachteile	gemeinsame Zustandsinfo teuer (durchreichen; Fehlerbehandlung schwer $\rightarrow$ <i>Fehlerkanal</i> einrichten); Effizienzgewinn oft geringer als vermutet (teure Übertragung, Synchronisation nicht trivial $\rightarrow$ überflüssige Transformationen vermeiden, Filter zusammenfassen).

9.3 Mediator-Pattern (Event-Driven Architecture)

Merke

Pipes & Filters in ereignisgesteuerten Architekturen: hochskalierbar, anpassungsfähig, Komponenten entkoppelt, Ereignisse *asynchron*. Ein **Tee-and-Join-Pipeline-System mit expliziten Pipes**. Komplexe Events werden vom **Mediator** aufgespalten in eigenständige, asynchron bearbeitbare Aufgaben.

Voraussetzung: *Bearbeitungsreihenfolge ist frei* $\Rightarrow$ Filter & Pipes austauschbar; nicht das Produkt, sondern die *Aktionen* stehen im Vordergrund. Verarbeitende Filter werden zu **Event-Prozessoren** zusammengefasst.



Der Mediator verteilt Aufgaben (Processing Events über spez. Pipes/Kanäle, *Publish-Subscribe* möglich); Event-Prozessoren kapseln die Geschäftslogik. **Vorteile:** anpassungsfähig, gute Performance (asynchron/parallel), individuell skalierbar. **Nachteile:** Abhängigkeit Mediator $\leftrightarrow$ Prozessor; neue Prozessoren erfordern Mediator-Anpassung. **Realisierung:** Message Broker oder Business-Process-Orchestration-Tools.

10 Middleware – gestern, heute, morgen

Merke

Die wenigsten Systeme entstehen „from scratch- meist Korrektur/Adaption/Erweiterung. Die **Middleware** ist das Framework dieser Umgebung: die *Abstraktionsebene zwischen Anwendung und realer Welt* (HW, OS, Cloud). Sie ist **unabhängig von der Applikation**, regelt Topologie (1-1, 1-n, n-m), bietet einheitliche Kommunikation/Datenaustausch, eine Laufzeitumgebung und ist für den Anwender *transparent*. „Das Rückgrat jeder Anwendung ist die richtige Middleware.“

10.1 Abstraktionsebenen

Gestern	Heute	Aufgabe
Transport Mechanism (OOM/RPC, MOM, CORBA, .NET Remote)	Transport & Networking (HTTP, gRPC, Kafka, MOM)	Kommunikation vereinheitlichen/vereinfachen
Application Server (JEE, Tomcat, Geronimo)	Services & Runtime (leichtgewichtig) (Spring Boot, Node.js)	Ausführungsumgebung; Aufteilung in unabhängige Services
Message Broker (ESB, Mule, WebSphere)	Container (Docker)	Anwendungen integrieren bzw. portabel/reproduzierbar bereitstellen
Business Process Orchestration (BPEL, BIZTALK)	Orchestration (Kubernetes)	Abläufe vereinheitlichen bzw. verteilen, ausfallsicheren Betrieb

Merke

Was hat sich geändert? Die klassischen, schweren Frameworks sind verschwunden – die *Features* nicht, sie sind in spezialisierte Komponenten gewandert: statt Magie mehr Transparenz, statt einer Lösung mehr Flexibilität & Kombinierbarkeit. Heutige App-Server bieten z. B. keine Verzeichnisdienste mehr, nur noch leichtgewichtiges Transaktionsmanagement.

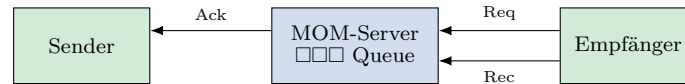
10.2 Kommunikation: OOM vs. REST/gRPC

	Klassisch: OOM	Modern: REST/gRPC
Ausführung	zustandsbasiert	zustandslos (funktional)
Programmiermodell	rein objektorientiert	service-/ressourcenorientiert
Aufrufe	i. d. R. synchron	synchron <i>oder</i> asynchron (kontextabh.)
Konfiguration	hochkomplex (IDL, ORB)	einfache Protokolle/Konventionen (REST/JSON)
Transferkosten	hoch (Parameter/Resultate)	bleiben

10.3 MOM – Entkopplung durch Nachrichten

Merke

Message-Oriented Middleware: Anwendungen sind autonom, kommunizieren über *Nachrichten*, die in **Warteschlangen** verwaltet werden. Die Messaging-Infrastruktur (MOM-Server) **entkoppelt Sender und Empfänger**. Queues werden mit ihren Empfängern assoziiert.



Aufgaben MOM-Server: Nachrichten annehmen & in richtige Queue einsortieren, Bestätigung an Sender, vorhalten bis abgeholt, auf Anfrage ausliefern. **Szenarien:** *Fire-and-Forget* (keine Bestätigung, z. B. Werbung); asynchron bei hohen Ausfallzeiten (SMS, WhatsApp, RabbitMQ).

QoS-Abstufungen (Reliability ↔ Performance, gegenläufig):

QoS-Stufe	Eigenschaft
Best Effort	nur im RAM, nach Zeit verworfen – hohe Performance.
Persistent	Auslieferung über Systemausfall hinaus garantiert (nicht-flüchtig) – geringe Performance.
Transactional	mehrere Nachrichten als Transaktion mit ACID – noch geringere Performance.

Ausfallsicherheit/Skalierung: *Cluster* (mehrere Server, gleiche Queues, statische/dynamische Zuteilung) ⇒ Lastverteilung + Failover. **Publish-Subscribe:** aus *Queues* werden *Topics*, aus 1-1 wird n-m – Empfänger = Subscriber (registrieren sich für Topics), Sender = Publisher; Server liefert an *alle* Subscriber. Duplex-Kanal nur über Nachrichteninhalte (Sender übermittelt eigene Empfänger-Queue).

Quality Attr.	MOM
Verfügbarkeit	physische Queues replizierbar → Ausfallsicherheit.
Modifizierbarkeit	Queues trennen Sender/Empfänger vollständig (nur Austauschformat koppelt).
Performance	QoS-abhängig; Netzwerk auf Nachrichtengröße/Aufkommen abstimmen.
Skalierbarkeit	Queues auf dedizierten Maschinen, replizierbar; Engpass ist das Netzwerk.

10.4 Gedanken zur Verteilung

Merke

Eine Middleware *reduziert nicht* den Kommunikationsaufwand! Ein verteiltes System ist mehr als die Verteilung von Objekten. Schnittstellen müssen nicht nur zur Aufgabe, sondern auch zum *Ausführungsmodell* passen.

First Law of Distributed Object Design: **Don't distribute your objects!** (Fowler) – Verteilung hat stets einen Grund.

Lokales Interface (feingranular): viele kleine Methoden, individuelle Getter/Setter.

Remote Interface (grobgranular): wenige Methoden, Daten kombiniert/komprimiert. $\Rightarrow$ Kommunikationsaufwand minimieren – die Flexibilität darf darunter leiden.

11 Patterns für den Business Layer

Es gilt, die Objekte des Anwendungsgebiets zu organisieren und mit den NFR (Performance, Flexibilität, Skalierbarkeit) in Einklang zu bringen; Verwaltungs-/Hilfskomponenten müssen integriert werden (Fowler).

11.1 GRASP – das Herzstück des Designs (Larman)

Verantwortlichkeiten werden den Objekten im Design zugewiesen:

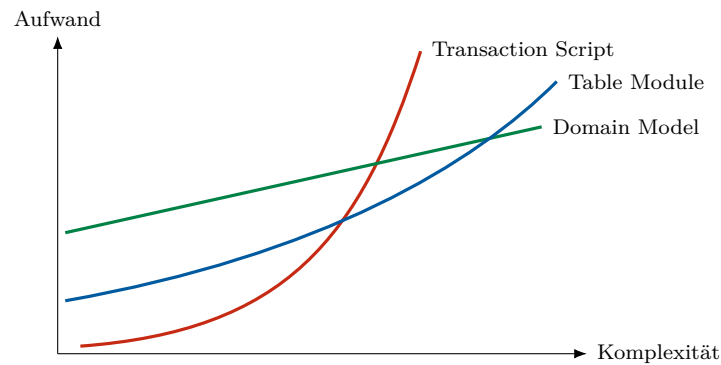
Verantwortlichkeit	Bedeutung
Verantwortung, etwas zu tun	selbst tun oder delegieren.
Verantwortung, etwas zu wissen	private Info kennen, abgeleitete/berechnete Info kennen, zugeordnete Objekte kennen.

Zuweisungsregeln: *Experte* (Verantwortung an die Klasse mit den nötigen Informationen), *lose Kopplung* (nicht jeder kennt jeden), *hohe Kohäsion* (nicht einer macht alles).

11.2 Die drei Business-Layer-Patterns

Pattern	Idee / Eigenschaften
Transaction Script	Geschäftslogik als definierte Methoden, je <i>Request/Use Case</i> eine Methode; oft direkter DB-Zugriff. <i>Einfach</i> , kaum Overhead. Nachteil: bei steigender Komplexität Übersichtsverlust, Code-Verdopplung, gemeinsame Funktionalität schwer auslagerbar. Ideal für einfache Anwendungen mit wenig Logik.
Domain Model	zentrale Teile des Problemraums in den Lösungsraum überführt; Aufgaben dort, wo sie anfallen („Simulation der Realität“). Eigener Domain-Layer; Daten + Funktionalität verbunden (Experte/lose Kopplung/hohe Kohäsion). Eine Klasse $\approx$ eine DB-Tabelle. Nachteil: Lifecycle/Session-Info, überladene Klassen (Use-Case-Logik nicht zwingend ins Domänenobjekt; verschiedene Interfaces nutzen).
Table Module	direkte Abbildung der relationalen Sicht: je Tabelle/View/Query eine Klasse, die ihre Funktionalität kapselt (Objekte repräsentieren Datenbankabfragen). Ideal für kleine, datenbankorientierte Anwendungen. Nachteil: komplexe Logik schwer auf Tabellen verteilbar; Views/Assoziationstabellen $\rightarrow$ Inkonsistenzen.

11.3 Aufwand im Verhältnis zur Komplexität (Fowler)



Klausur-Fokus

Transaction Script ist bei geringer Komplexität am günstigsten, explodiert aber. **Domain Model** hat hohen Grundaufwand, skaliert aber flach mit der Komplexität. **Table Module** liegt dazwischen. Tipp: *vielen realen Problemen liegen auf der Skala links* (geringe Komplexität).

12 Zwischen Architektur und Design: Interfaces

Entwurfsmuster sind im Kleinen wie im Großen bewährte Lösungen wiederkehrender Probleme – sie setzen **nicht-funktionale Requirements auf Code-Ebene** um.

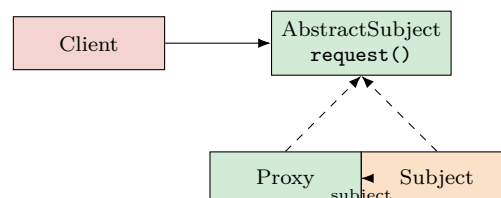
Merke

Ein gelungener Entwurf trennt **Interface** und **Implementierung** sowie **Allgemeingültiges** (stabil) und **Spezifisches** (variabel) und verbirgt eine spezifische Implementierung hinter einem allgemeinen Interface ($\Rightarrow$ Spezifisches wird austauschbar).
Leitsatz (GoF): *Man programmiert auf eine Schnittstelle hin, nicht auf eine Implementierung* – Clients kennen nur Interfaces, nicht die konkreten Klassen.

12.1 Proxy

Merke

Ein **Proxy** ist Platzhalter für ein echtes Objekt und *kontrolliert den Zugriff*. Proxy und Original (Subject) haben *dasselbe Interface*; der Proxy fügt i. d. R. keine neue Funktionalität hinzu, sondern erleichtert/verbessert den Zugriff (`request()` delegiert an `subject->request()`).



Dynamik: `Client.service()` $\rightarrow$ `Proxy.vorbereiten()` $\rightarrow$ `Subject.service()` $\rightarrow$ `nachbereiten()`.

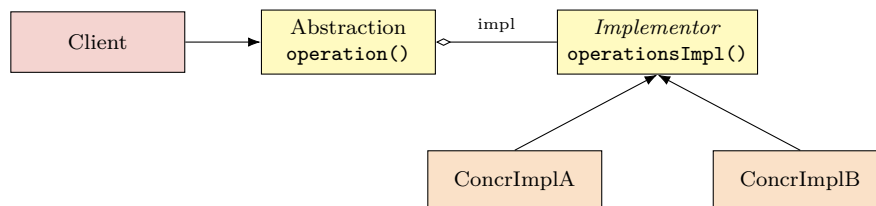
Proxy-Art	Zweck
Virtueller Proxy	Lazy Evaluation (Objekt erst bei Bedarf erzeugen).
Schutz-Proxy	überwacht den Zugriff (Berechtigung).
Cache-Proxy	vermeidet unnötige Kommunikation.
Synchronisations-Proxy	nach Original „thread-safe“ (Monitor, <code>synchronized</code>).
Reverse Proxy	vertritt den <i>Client</i> (Agent): Caching/Maskieren von Abbrüchen, Antworten optimieren (komprimieren/filtern), Single Sign-on, Lastverteilung, Anonymisierung.

Konsequenz: zusätzliche Indirektionsstufe für Anpassungen. **Anwendbarkeit:** bei Bedarf einer anpassungsfähigen, „intelligenten“ Referenz auf ein Objekt/eine Komponente (Anknüpfungspunkt für Adaptionsmechanismen).

12.2 Bridge (Brücke)

Merke

Die **Brücke** trennt *Abstraktion* (Interface) und *Implementierung*, sodass beide **unabhängig** variieren. Die Abstraktion definiert das Interface und delegiert an einen Implementor (`operation() → impl->operationsImpl()`).



Anwendbarkeit: permanente Bindung Abstraktion↔Implementierung vermeiden (beide mehrfach kombinierbar); Austausch ohne Einfluss auf den Client (kein Neuübersetzen); Implementierung vollständig verbergen.

Beispiel Symboltabelle als Brücke

Dictionary (abstract, Interface `insertNew/lookup`) delegiert an DictionaryImpl. Symtable erweitert Dictionary; die konkrete DictionaryImpl (z. B. DictionaryHtab mit HashMap) bleibt verborgen und ist später austauschbar. **Als Interceptor:** Client-Side-Agent (Abstraktion) + Server-Side-Agent (Implementor) realisieren beliebige Adaptionsmechanismen.

12.3 Adapter

Merke

Der **Adapter** passt eine *bestehende* Schnittstelle an die Bedürfnisse eines Clients an, damit Komponenten mit inkompatiblen Schnittstellen zusammenarbeiten. Ein **Objekt-Adapter** nutzt *Komposition + Delegation* (`operation() → specObj.specOperation()`).

Einsatz: existierende Komponente passt nicht; Komponente entsteht, bevor übergeordnete Schnittstellen feststehen; austauschbare Komponenten ohne praktikable gemeinsame Schnittstelle.

Adapter-Art	Eigenschaft
Zweiweg-Adapter	bietet beide Schnittstellen (Ziel <i>und</i> Adaptee) an.
Pluggable-Adapter	generisch, bildet eine Schnittstelle per <code>bind(self, adaptee)</code> (z. B. via Reflection) auf andere ab – Komponenten von Anfang an adaptierbar.

Intuition – einfach erklärt

Proxy vs. Adapter vs. Bridge: Der *Proxy* behält das *gleiche* Interface und kontrolliert nur den Zugriff. Der *Adapter* *ändert* ein vorhandenes Interface in ein gewünschtes. Die *Bridge plant von Anfang an* die Trennung von Interface und Implementierung, damit beide unabhängig wachsen.